



Maximum Profit by Selling K Tickets (Greedy → Heap → Binary Search on Answer)

Problem

Given an array `arr[]` where:

- `arr[i]` = number of tickets seller `i` currently has.
- Price of every ticket = remaining tickets of that seller **at the time of selling**.
- After selling one ticket, seller's value decreases by 1.
- Sell **at most K tickets** to maximize profit.

Example

```
arr = [4,3,6,2,4]
k = 3
```

```
Answer = 15
```

```
Sell:
```

```
6
5
4
```

```
Total = 15
```

Step 1 : Understanding the Problem

Each seller behaves like

```
Seller having 6 tickets
```

```
6
↓
```

```
5
↓
```

```
4
↓
```

```
3
```

↓

2

↓

1

↓

0

Each sale decreases only **that seller**.

Example

Initial

4 3 6 2 4

Sell from 6

4 3 5 2 4

Sell again

4 3 4 2 4

Observation

At every moment

We should always sell the **highest priced ticket available**.

Why?

Suppose current prices

8 6 4

Selling

4

before

8

never increases profit.

Hence

Always sell the maximum available ticket.

This is the Greedy Choice.

Brute Force

Idea

Repeat K times

```
Find maximum seller  
Sell one ticket  
Decrease seller by one
```

Algorithm

```
repeat k times  
    find maximum element  
    answer += maximum  
    maximum--
```

Code

```
def maxProfit(arr,k):  
    ans=0  
    while k>0:  
        idx=0
```

```
    for i in range(1,len(arr)):
        if arr[i]>arr[idx]:
            idx=i

    if arr[idx]==0:
        break

    ans+=arr[idx]

    arr[idx]-=1

    k-=1

return ans
```

Dry Run

```
arr

4 3 6 2 4
```

Iteration 1

```
take 6

profit=6

4 3 5 2 4
```

Iteration 2

```
take 5

profit=11

4 3 4 2 4
```

Iteration 3

```
take 4

profit=15
```

Answer

15

Complexity

Finding maximum

$O(n)$

Done

k

times

$O(n*k)$

Impossible for

$n=100000$

$k=1000000$

First Optimization : Max Heap

Question

What is expensive?

Finding maximum every time.

Can we always keep maximum ready?

Yes.

Use **Max Heap**.

Heap Logic

Initial

```
6
4
4
3
2
```

Pop maximum

```
6
```

Profit

```
6
```

Seller now has

```
5
```

Push back

Heap

```
5
4
4
3
2
```

Again

```
Pop 5
```

```
Profit=11
```

```
Push 4
```

Heap

4
4
4
3
2

Again

Pop 4

Profit=15

Done.

Heap Code

```
import heapq

def maxProfit(arr,k):

    heap=[-x for x in arr]

    heapq.heapify(heap)

    ans=0

    while heap and k:

        cur=-heapq.heappop(heap)

        if cur==0:
            break

        ans+=cur

        cur-=1

        if cur:
            heapq.heappush(heap, -cur)

        k-=1

    return ans
```

Complexity

Heap Build

$$O(n)$$

Every sale

$$\text{Pop} = O(\log n)$$

$$\text{Push} = O(\log n)$$

Total

$$O(n + k \log n)$$

Still too slow for

$$k = 10^6$$

because we still simulate every ticket.

Final Optimization

Binary Search on Answer

This is the difficult part.

Don't memorize.

Understand the thought process.

How do we build this idea?

Thought 1

Heap still sells

one ticket

one ticket

one ticket

...

Question

Can we sell many tickets together?

Answer

Yes.

Example

Seller

10

Need to sell until

7

Instead of

10

9

8

7

Use Arithmetic Progression

$10+9+8+7$

computed in $O(1)$.

Great.

Now another question.

Thought 2

Where should we stop?

Example

10

7

5

Should first seller stop at

7?

Or

6?

Or

5?

Unknown.

Need a better way.

Thought 3

Instead of selling tickets

Think in **Levels**

10

7

5

Reduce everyone above

Level = 6

Result

6

6

5

How many tickets sold?

Seller 1

10→6

4 tickets

Seller 2

7→6

1 ticket

Seller 3

5

0

Total

5 tickets

Notice

Without simulation

We computed

10-6

+

7-6

=

5

General Formula

`ticketsSold(level)`

=

$\sum \max(0, \text{arr}[i] - \text{level})$

One pass.

$O(n)$.

Thought 4

Let's try different levels

Example

10

7

5

k=5

Level

8

Tickets=2

Level

7

Tickets=3

Level

6

Tickets=5

Level

5

Tickets=7

Table

Level	Tickets Sold
8	2
7	3
6	5
5	7

Observe

As level decreases

Tickets sold increases.

Monotonic.

Whenever you see

Increasing X

always

decreases Y

or vice versa

Think

Binary Search on Answer

Binary Search

Search

Highest level

such that

```
ticketsSold(level)>=k
```

Search Space

0

to

max(arr)

Check Function

```
sold=0

for x in arr:
    sold+=max(0,x-mid)
```

If

```
sold>=k
```

Move Right

Else

Move Left

After Binary Search

Suppose answer

```
level=6
```

Every seller above 6

Sell

```
10  
9  
8  
7
```

Instead of loop

Use AP

```
Sum=(first+last)*count//2
```

where

```
first=10  
last=7  
count=4
```

If we sold

few extra tickets

Remove

```
extra*level
```

because extra tickets are exactly worth

```
level
```

Optimized Code

```
class Solution:
    def maxProfit(self, arr, k):
        MOD = 10**9 + 7

        low = 0
        high = max(arr)

        while low <= high:
            mid = (low + high) // 2

            sold = 0
            for x in arr:
                if x > mid:
                    sold += x - mid

            if sold >= k:
                low = mid + 1
            else:
                high = mid - 1

        level = high

        ans = 0
        sold = 0

        for x in arr:
            if x > level:
                cnt = x - level
                sold += cnt

                ans += (x + level + 1) * cnt // 2

        extra = sold - k
```



```
ans -= extra * (level + 1)
```

```
return ans % MOD
```

Dry Run

10

7

5

k=5

Binary Search finds

level=6

Seller1

10+9+8+7=34

Seller2

7=7

Total

41

Tickets sold

5

Done.

Recognition Pattern

Whenever you see

Repeatedly reduce
Repeatedly increase
Repeatedly consume
Repeatedly remove

Ask

Can I batch operations?

If Yes

↓

Think

Threshold
Cutoff
Level
Height
Capacity

↓

If answer changes monotonically

↓

Binary Search on Answer

Similar Problems

- Sell Diminishing-Valued Colored Balls (LeetCode 1648)
- Cutting Wood Problem
- Aggressive Cows
- Allocate Books

- Painter Partition
 - Capacity To Ship Packages
 - Koko Eating Bananas
 - Minimize Maximum Distance
 - Maximum Candies Allocation
 - Chocolate Distribution (Binary Search Variant)
-

Pattern Sheet

Brute Force

↓

Simulation

↓

Find expensive operation

↓

Replace repeated search using Heap

↓

Still simulating?

↓

Batch operations

↓

Represent batch using Threshold

↓

Can threshold be checked in $O(n)$?

↓

YES

↓

Is check monotonic?

↓

YES

↓

Binary Search on Answer

Recognition Checklist

- ✓ Repeated operations
- ✓ Same operation again and again
- ✓ Values decrease/increase gradually
- ✓ Need maximum/minimum answer
- ✓ Simulation too slow
- ✓ Can think in Levels instead of Elements
- ✓ Threshold possible
- ✓ Check function possible in $O(n)$
- ✓ Monotonic behaviour

→ Binary Search on Answer

Real World Analogy

Imagine several water tanks.

10L

7L

5L

Instead of removing

```
1 litre
```

```
1 litre
```

```
1 litre
```

from random tanks,

you lower all tanks to a common water level.

The **level** is what Binary Search finds.

Profit calculation becomes simple arithmetic instead of simulation.

Complexity Summary

Approach	Time	Space
Brute Force	$O(n \times k)$	$O(1)$
Heap	$O(n + k \log n)$	$O(n)$
Binary Search + AP	$O(n \log(\max(\text{arr})))$	$O(1)$

What I Learned Today

1. Greedy choice = Always sell highest current ticket.
2. Heap removes repeated maximum search.
3. Heap still simulates every ticket.
4. Think in **levels**, not tickets.
5. A level defines how many tickets are sold.
6. Ticket count from a level is monotonic.
7. Monotonic check $\Rightarrow$ Binary Search on Answer.
8. Arithmetic Progression computes profit in $O(1)$ instead of repeated simulation.

Golden Rule: When simulation is too slow, ask: *Can I replace individual operations with a common threshold or level and compute the result in batches?*